COSC 466/566 Spring 2026
Software (and Web) Security

Doowon Kim (doowon@utk.edu)
Assistant Professor of Computer Science
University of Tennessee, Knoxville

THE UNIVERSITY OF TENNESSEE KNOXVILLE

Bonus Quiz

- What does 0 say to 8?

Bonus Quiz

- What does 0 say to 8?

- "Nice belt!”

- Why?

Bonus Quiz

- What does 0 say to 8?

- "Nice belt!”

- Why?
  - 8 looks like 0 with a belt (or line) around the middle.

What Will Be Covered In This Class?

- Machine Programming 101
- Buffer Overflow Vulnerability
- Frame Pointer Attack
- Writing Shell Code
- Mitigations: Dep, Stack Cookie, Address Space Layout Randomization (ASLR)
- Return Oriented Programming
- Format String Vulnerability
- How can we use LLMs for Software Security?

What Will Be Covered In This Class? (2)

- Web + DB 101
- SQL Injection Attack
- Cookie
- CSRF + XSS Attack
- How can we use LLMs for Web Security?

Security Mindset

Security Mindset

- Security requires a particular mindset
  - i.e., Security Mindset
- Involves thinking about how things can be made to fail
  - E.g., attacker or criminal

Security Mindset

- To anticipate attackers, we must be able to think like attackers
- The ability to view a large, complex system and be able to reason about:
  - What are the potential security threats?
  - What are the hidden assumptions?
  - Aare the explicit assumptions true?
  - How can we mitigate the risks of the system?

Security Mindset (example)

- An automobile dealership.
- She was able to retrieve her car after service just by giving the attendant her last name.
- Now any normal car owner would be happy about how easy it was to get her car back,
- but someone with a security mindset immediately thinks: "Can I really get a car just by knowing the last name of someone whose car is being serviced?"

Security mindset example

- Any example that you want to share today?

What is Computer Security?

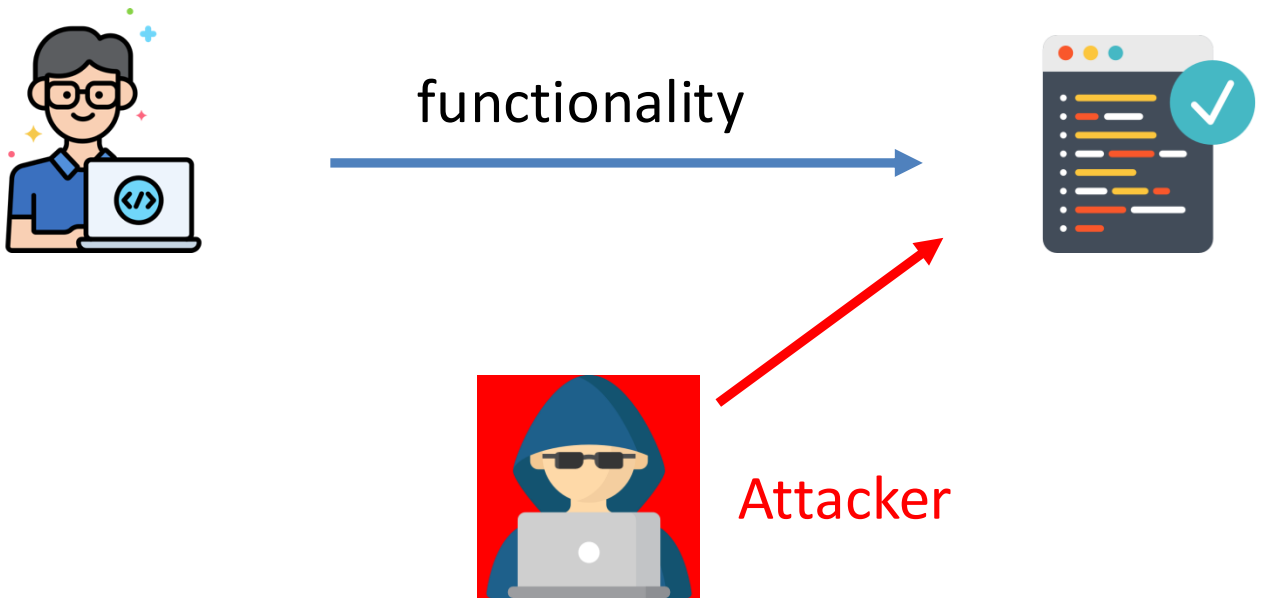
- Normally, we are concerned with functionality and correctness.

- Security is also a form of correctness.

What is Computer Security?

The key difference:

- Security involves an adversary who is active and malicious.
- Attackers seek to circumvent protective measures.



What's the Diff. between You and Attackers?

- Attackers are not normal users

- Normal users: try to avoid bugs/flaws
- Attackers: try to find the bugs/flaws out and to exploit them

```
//Check if enough money in account to handle the charge, if so execute
public boolean chargeAccount(double debitAmount){
        if(this.balance >= debitAmount){
                this.balance -= debitAmount;
            return true;
        }else{
                return false;
        }
}
```

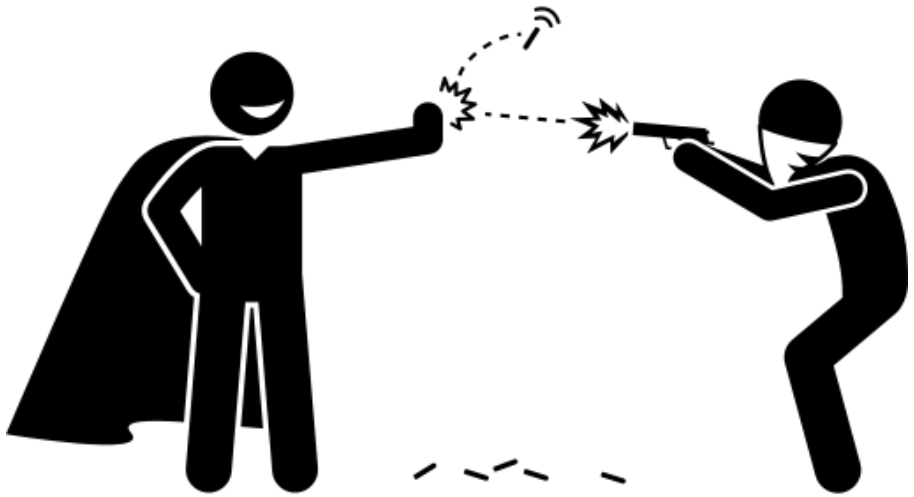
What is the assumption?
What can an adversary do with it?

```
//Check if enough money in account to handle the charge, if so execute
public boolean chargeAccount(double debitAmount){
        if(this.balance >= debitAmount){
                this.balance -= debitAmount;
            return true;
        }else{
                return false;
        }
}
```

myself.chargeAccount(-100.00);

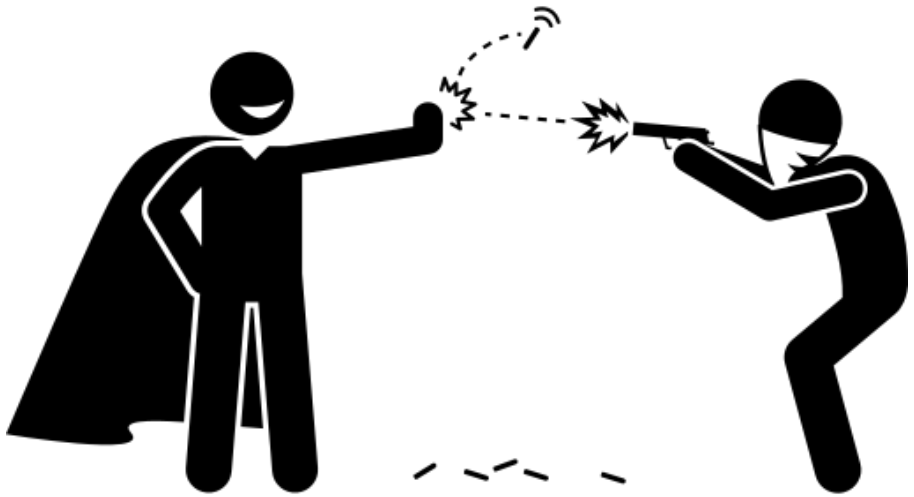
Attacker ←→ Defender

Attackers Vs. Defenders



Defenders **Attackers**

Attackers Vs. Defenders (Incorrect View)



Defenders

Mutual Conflict

Attackers

Attackers Vs. Defenders (Real-world)

Attackers

Detect

Evade & Bypass

Defenders

Phish/Deceive Users

Find & Exploit Bugs

...

Educate

Fix the bugs

...

Real-world Users

What does it mean to be secure?

- There is no such thing as security, only degrees of insecurity.

- Goal: just raise the bar for the attacker
  - Too difficult
  - Too expensive

- Ultimately, we want to mitigate undesired behavior

Basic Security Principles

- **Confidentiality**: an attacker cannot *recover protected data*.
  - Information Leak (e.g., side channel)
- **Integrity**: an attacker cannot *modify protected data*.
  - Data Tampering
- **Availability**: an attacker cannot *stop/hinder computation*.
  - Denial of Service Attack

- *These fundamental CIA properties are used in different contexts of software security policies and mechanisms*

Security Analysis

- Given a software system, is the system secure?
- *No simple answer, it depends on*
  - What is the interface? (What is the attack surface?)
    - Amazon Echo: Voice

Dolphin Attack

- Inaudible ultrasound commands can be used to secretly control Siri, Alexa, and Google Now

Dolphin Attack

Security Analysis

- Given a software system, is the system secure?
- *No simple answer, it depends on*
  - What is the interface? (What is the attack surface?)
    - Amazon Echo: Voice
    - Camera (e.g., Self-driving cars): Video
    - …
  - What are the assets? (How profitable is an attack?)
    - Hacking Amazon accounts: Credit cards information, personal info.
    - Hijacking Internet Sessions (e.g., eBay): Can buy things, but they will be shipped to me.
    - …
  - What are the goals? (What drives an attacker?)
    - State-funded attackers (Advanced Persistent Threat)
    - Money (Sell personal information)
    - Leverage compromised machines to launch other attacks (e.g., DDoS)
    - …

Threat Model

- **Threat Model: Define an attacker's abilities and resources.**
- Precisely describe who you are dealing with and how capable the bad guys (i.e., attackers) are.
- Example: You have a system that protects a web server
  - An attacker is remotely accessing the web server
  - An administrator of the web server is trusted
  - An attacker may intercept connections between client users and the web server (e.g., Man-in-the-Middle attack)

Cost of Security

- A security system is
  - expensive to develop (man month – X developers for Y months),
  - may incur performance and space overhead,
    - Adding more code will incur performance overhead
    - Changing data structures may incur performance overhead too because it affects cache behaviors of CPU
  - may be inconvenient for users.
    - User Access Control – from Windows Vista
- A practical security solution is
  - less expensive to develop (general solution – resilient to software/system updates, not an ad-hoc patch)
  - low performance and space overhead (less impact on data structures)
  - easy to use.

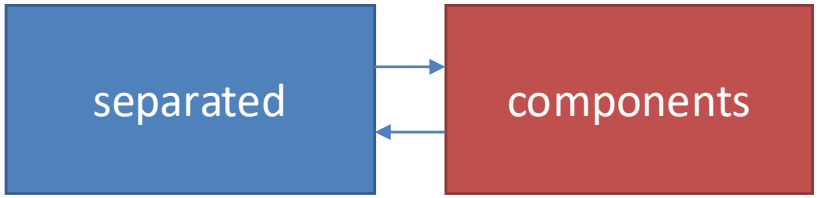
Security Concepts

- **Least Privilege:**
  A component (e.g., a process, a user, or a program, depending on the subject) must be able to access only the information and resources that are necessary for its legitimate purpose (i.e., least information is granted for the purpose).

- Any privilege that is further removed from the component will break the legitimate functionality (purpose).
- *No additional privilege is needed* for all functionalities of the components.

Security Concepts

- **Separation**
  - Separate individual components into smallest sub-components possible. The sub-components will communicate with each other.
  - Each small component will run with the least privilege.
  - At the boundary of each component, permissions will be checked.
- **Isolation vs. Separation**
  - Two components are separated: They do not overlap at all. They are completely separated.
  - Isolation does not necessarily mean that they are separated (Error-prone). https://en.wikipedia.org/wiki/Security_and_safety_features_new_to_Windows_Vista#Application_isolation



Security Concepts

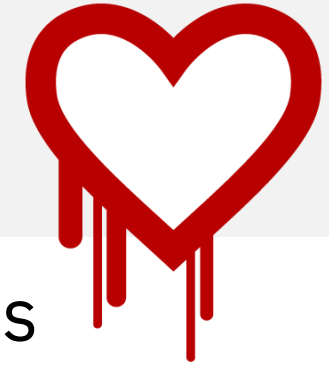
- **Trust:** Some components are assumed to be trusted
  - Applications trust OS (i.e., kernel)
  - OSes trust hardware
- **Correctness:** *Formal verification* ensures that if a component *correctly* implements a verified specification that can be trusted.
  - Informally, formal verification explores all possible ways including every single corner cases to see whether predefined properties (e.g., does this component leak secret information?) are satisfied.
  - In practice, implementations are often not correct.

Security Implementations

- Each process in modern operating systems (e.g., Windows/Linux) has their own memory space and is not supposed to access memory.
- Enforced by Virtual Memory through hardware:
  - In x86, each process maintains a page table address on the CR3 register (which is a table of addresses of memory pages that belong to the process).
  - When the CPU accesses memory, it looks up the table according to the address stored in the CR3 register.
  - CR3 can be only changed by the kernel.
- DOS and Windows 3.1 ***DID NOT*** have this luxury – What could go wrong? A memory bug in one application on Windows 3.1 can crash the entire OS.
- We are using Windows 10: All good? Of course no.
  - CreateRemoteThread – Creating a thread on another process.
  - Library Injection through Window Event Hooking

Example: Heartbleed Bug

Example: The Heartbleed Bug

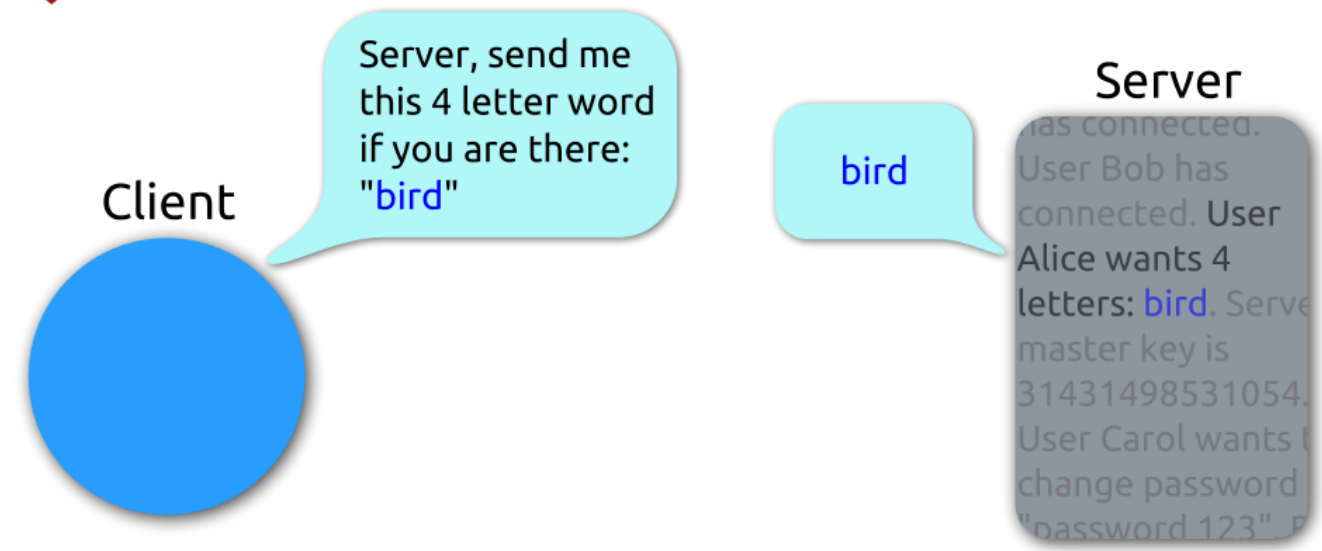


- A security bug in the OpenSSL cryptography library, which is widely used implementation of the Transport Layer Security (TLS) protocol **[wikipedia]**
- TLS is the de facto protocol for secure online communication
- This weakness allows stealing the information
  - user names & passwards
  - instant messgaes & emails
  - business critical documents & communication
  - more..

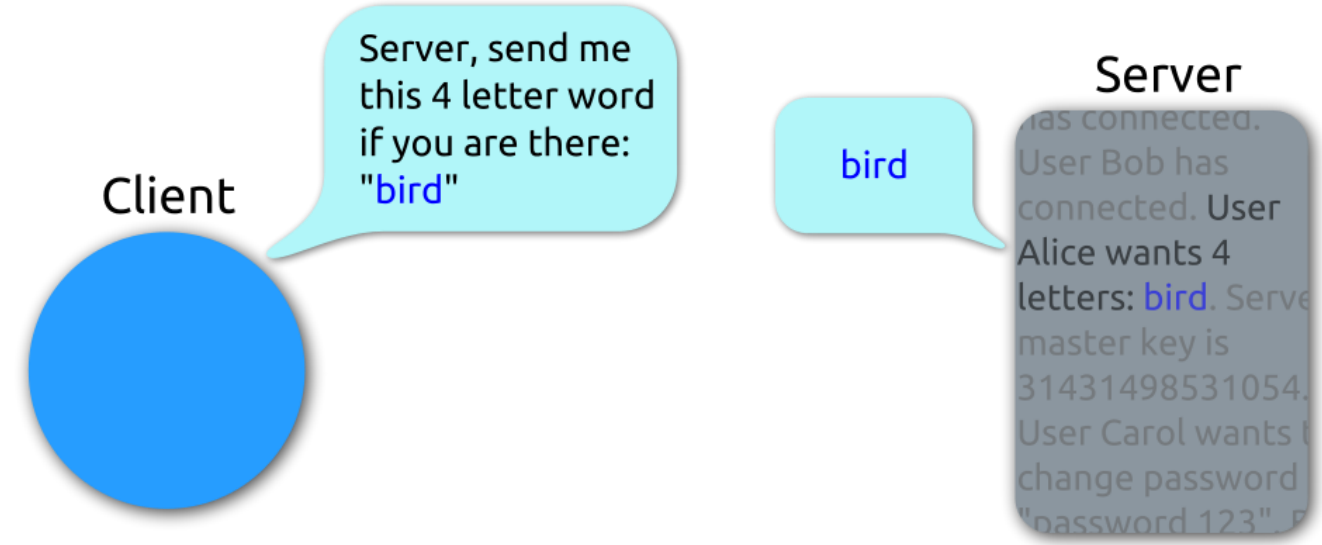
Example: The Heartbleed Bug

- The vulnerability is a “buffer over-read”
  - Software (accidently) allows more data to be read than should be allowed

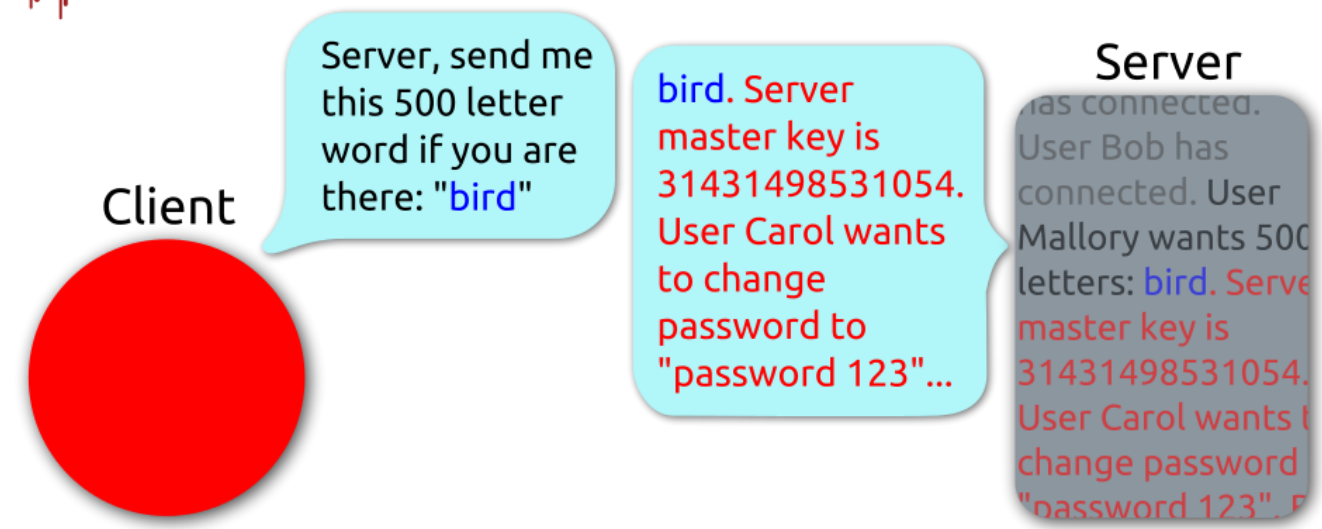
Heartbeat – Normal usage



Heartbeat – Normal usage



Heartbeat – Malicious usage



Example: The Heartbleed Bug

- The vulnerability is a “buffer over-read”
  - Software (accidently) allows more data to be read than should be allowed
- It is a simple programming error, but it is hard to discover
  - Vulnerable OpenSSL was released on March 14, 2012
  - Google’s security team reported Heartbleed on April 1, 2014

How to find a problem?

- Manual program inspection
  - Maybe effective
  - But humans are not good at
    - Repetitive and tedious tasks
    - Maintaining large amounts of detail
- Automated program analysis
  - Replace human inspection to find software problems
  - Support inspection by
    - Automated extracting and summarizing information
    - Automatically analyze extracted information
  - Large Language Models (LLMs)

Example: Vulnerability Detection in Code

```
from django.http import HttpResponse
from django.views.decorators.csrf import csrf_exempt
from django.core.files.storage import default_storage

@csrf_exempt
def upload_file(request):
    if request.method == 'POST':
        file = request.FILES['file']
        file_name = default_storage.save(file.name, file)
        file_url = default_storage.url(file_name)

        return HttpResponse(file_url)
```

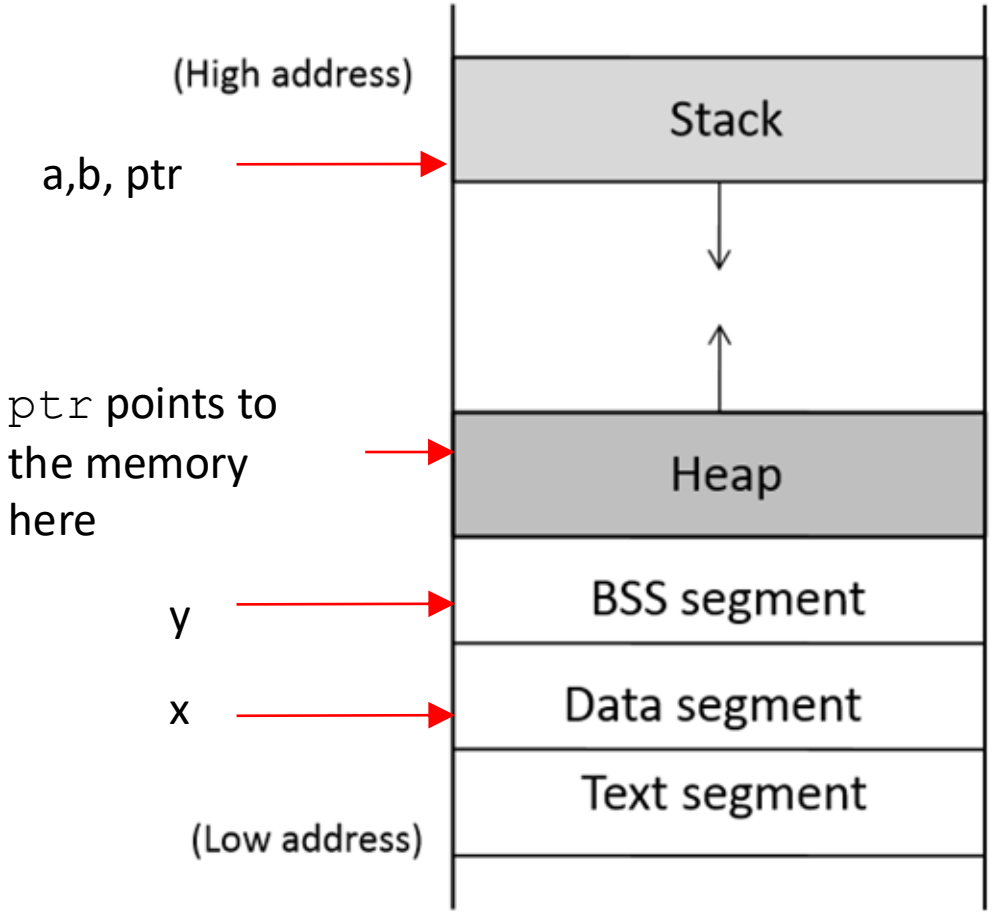
Example: Vulnerability Detection in Code

- Validate the file extension. Only allow safe extensions and disallow double extensions.
- Implement server-side validation to check if the file contains any malicious code.
- Limit the file size to prevent large file uploads which can lead to Denial of Service (DoS) attacks.
- Store the uploaded files in a separate directory that doesn't have execute permissions, to prevent any potential code in the file from being executed.

Overview: Stack Overflow

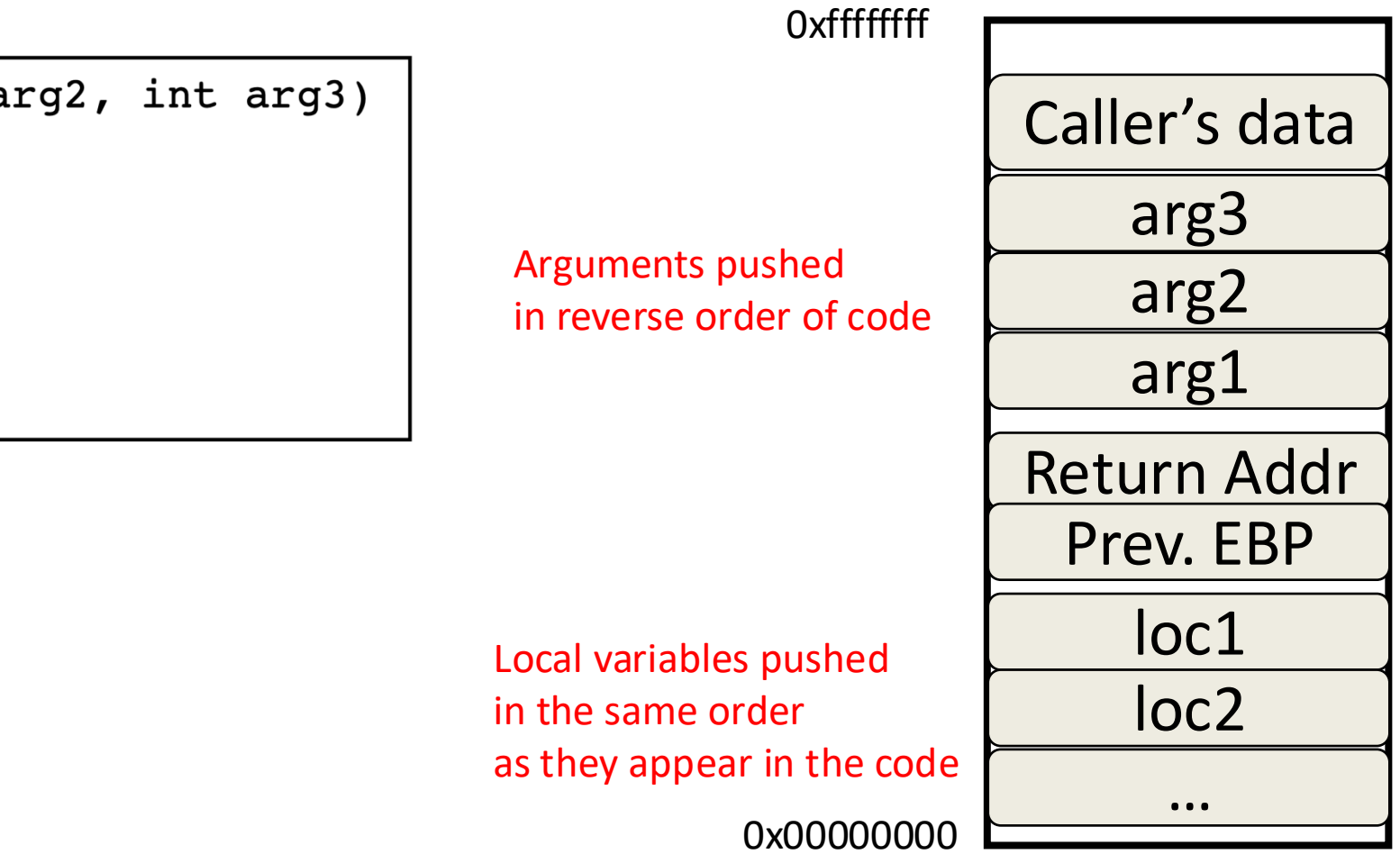
Program Memory Stack

```
int x = 100;
int main()
{
   // data stored on stack
   int   a=2;
   float b=2.5;
   static int y;

   // allocate memory on heap
   int *ptr = (int *) malloc(2*sizeof(int));

   // values 5 and 6 stored on heap
   ptr[0]=5;
   ptr[1]=6;

   // deallocate memory on heap
   free(ptr);

   return 1;
}
```

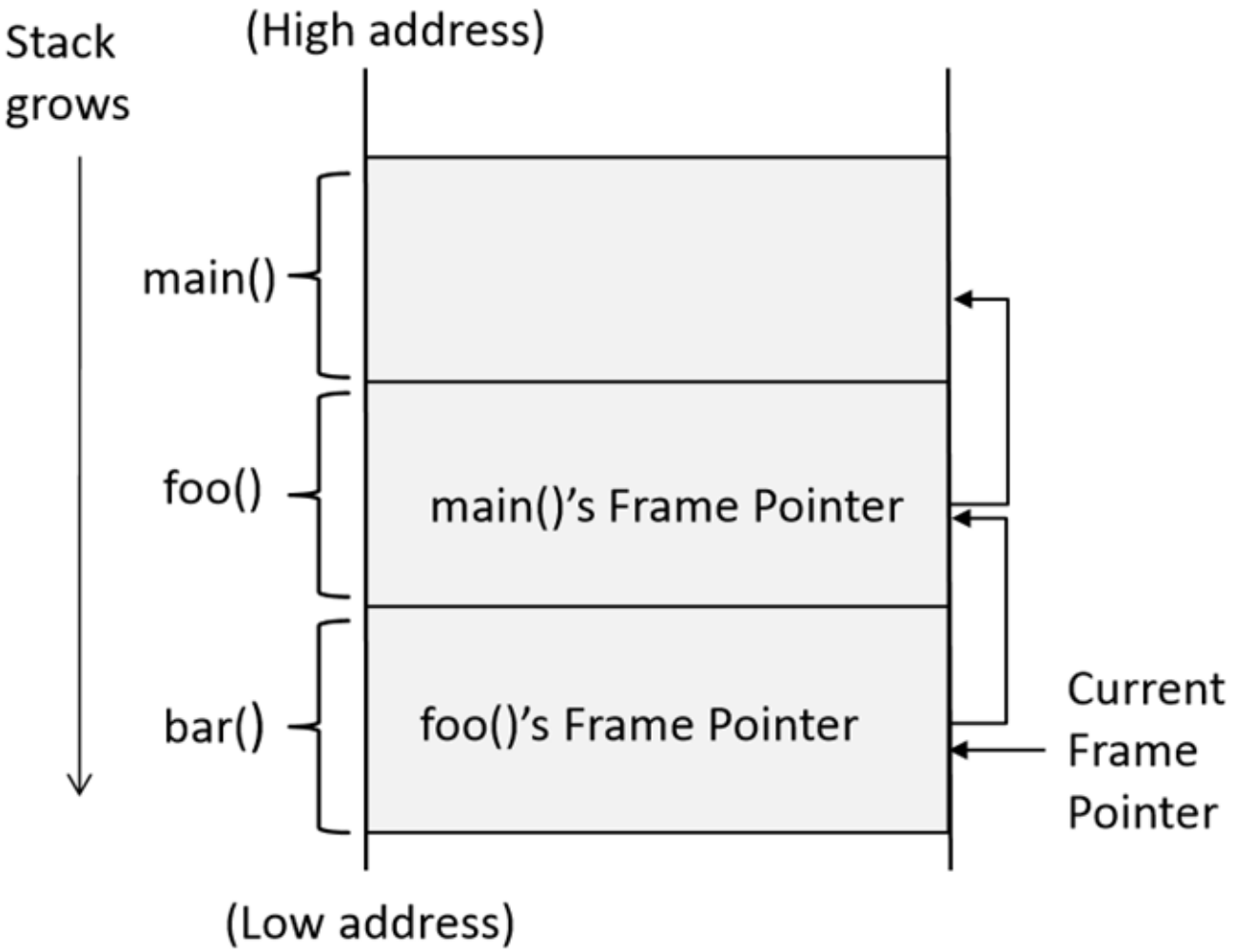


Stack layout when calling function

```
void func(char *arg1, int arg2, int arg3)
{
    char loc1[4]
    int  loc2;
    int  loc3;
    ...
}
```



Stack Layout for Function Call Chain



main()

foo()

bar()

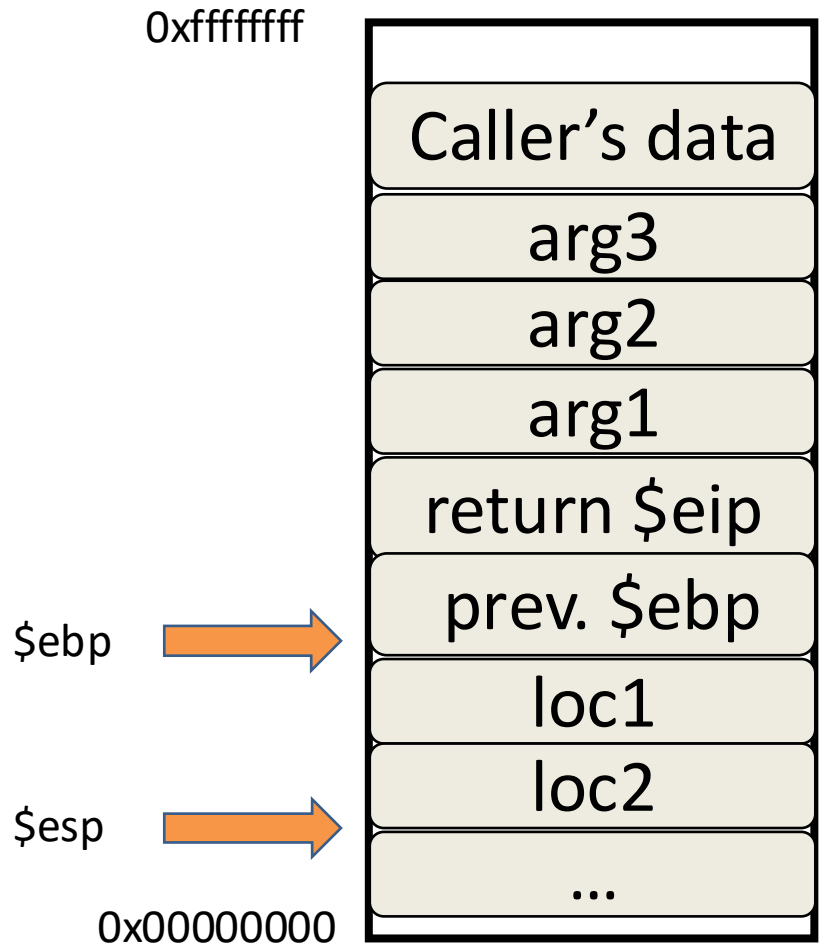
Returning from functions

In C

```
return;
```

In compiled assembly

```
leave:  mov %esp %ebp
        pop %ebp
ret:    pop %eip
```



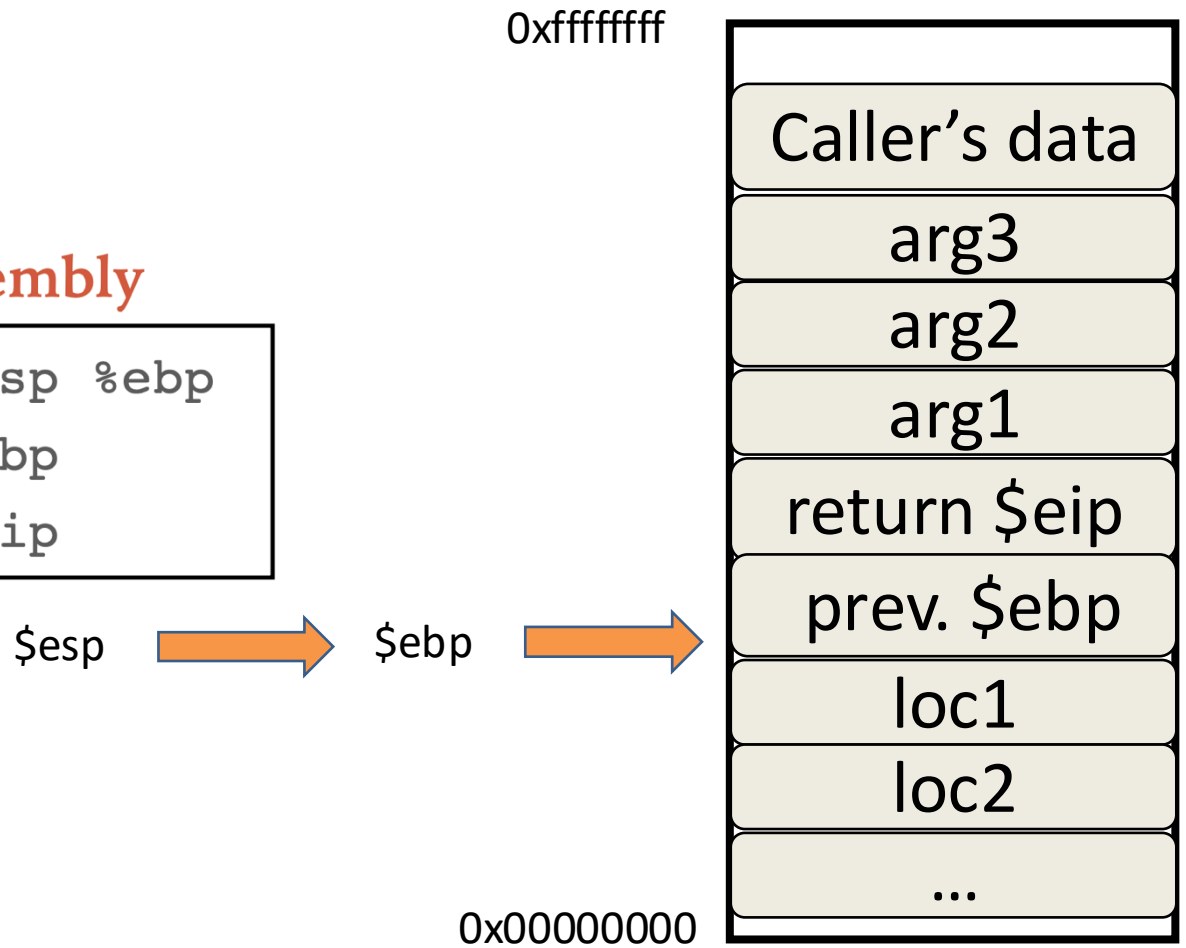
Returning from functions

In C

```
return;
```

In compiled assembly

```
leave:  mov %esp %ebp
        pop %ebp
ret:    pop %eip
```



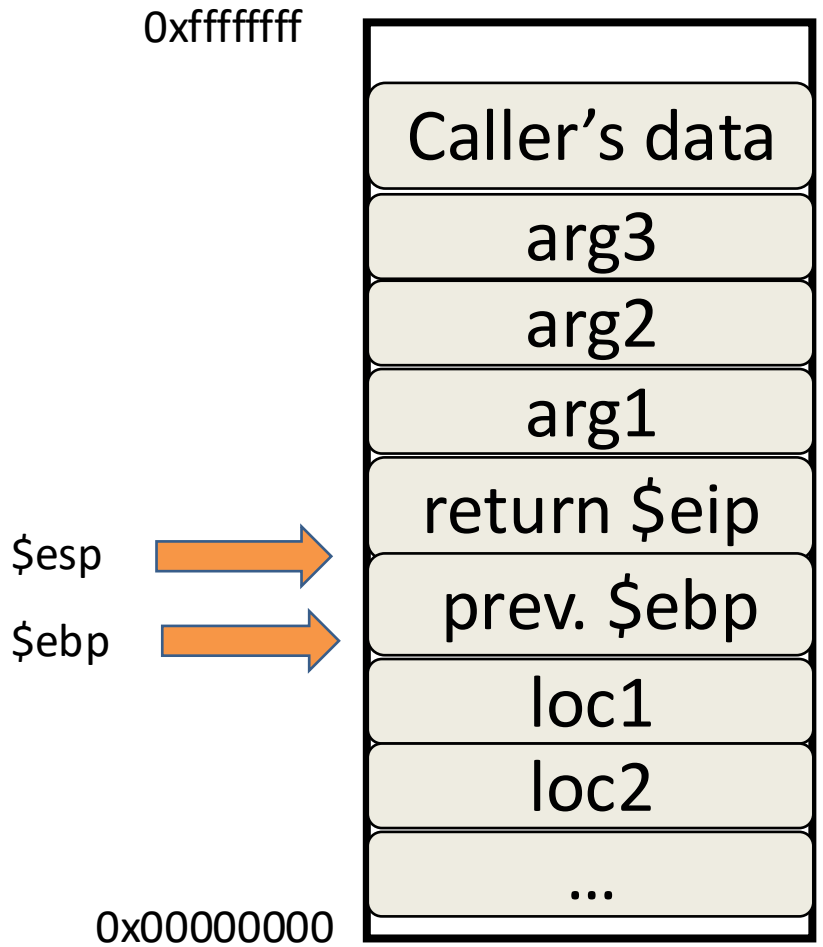
Returning from functions

In C

```
return;
```

In compiled assembly

```
leave:   mov %esp %ebp
         pop %ebp
ret:     pop %eip
```



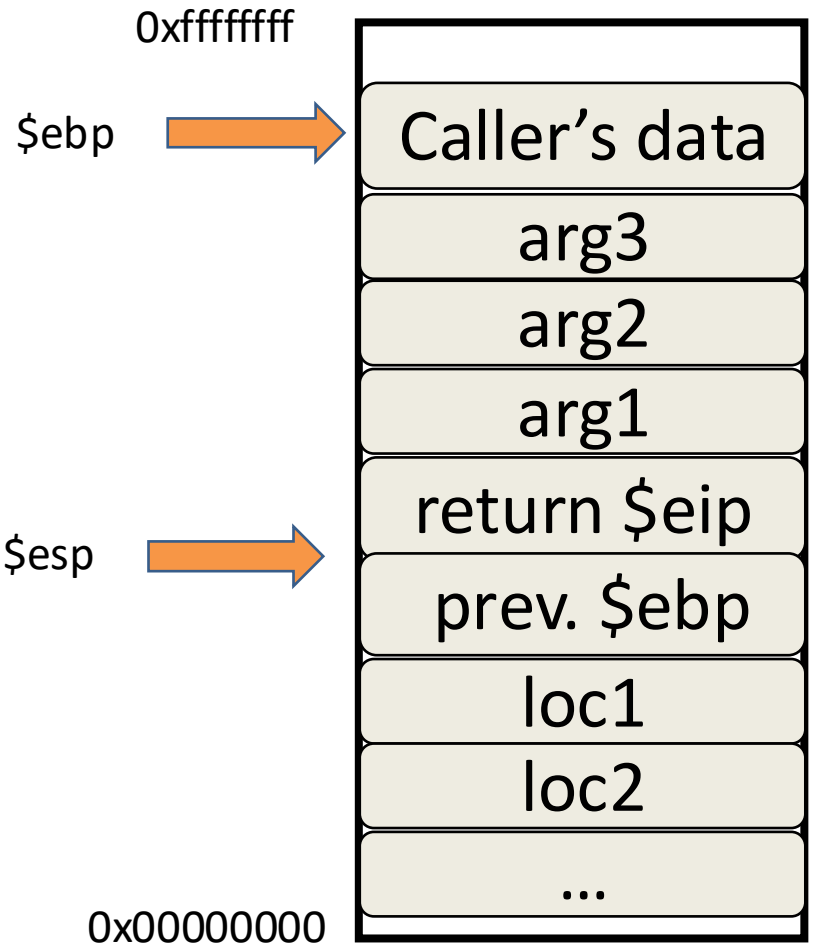
Returning from functions

In C

```
return;
```

In compiled assembly

```
leave:   mov %esp %ebp
         pop %ebp
ret:     pop %eip
```



Returning from functions

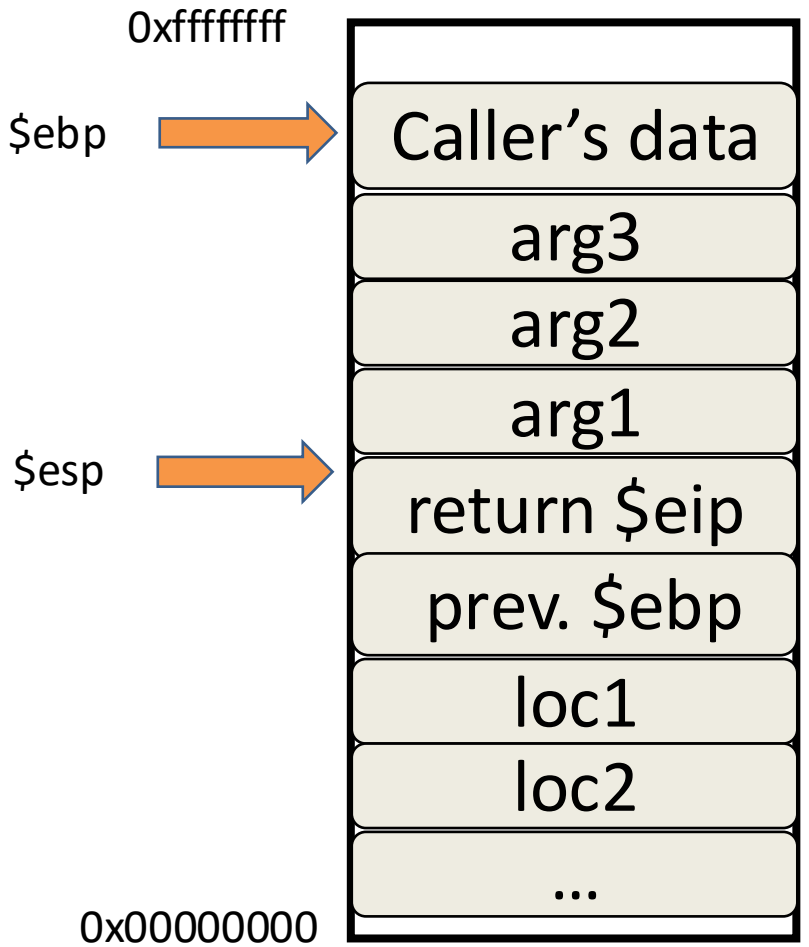
In C

```
return;
```

In compiled assembly

```
leave:   mov %esp %ebp
         pop %ebp
ret:     pop %eip
```

1. The next instruction is to “remove” the arguments off the stack
2. And now we’re back where we started



Common functions that cause overflow

```c
#include <string.h>
void foo(char *str)
{
    char buffer[12];
    /* The following statement will result in a buffer overflow */
    strcpy(buffer, str);
}
int main() {
    char *str = "This is definitely longer than 12";
    foo(str);
    return 1;

}
```